

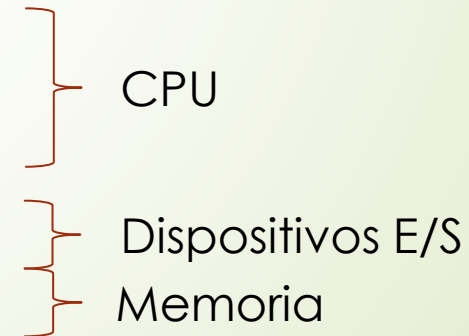


Tema 1: Estructura de la máquina

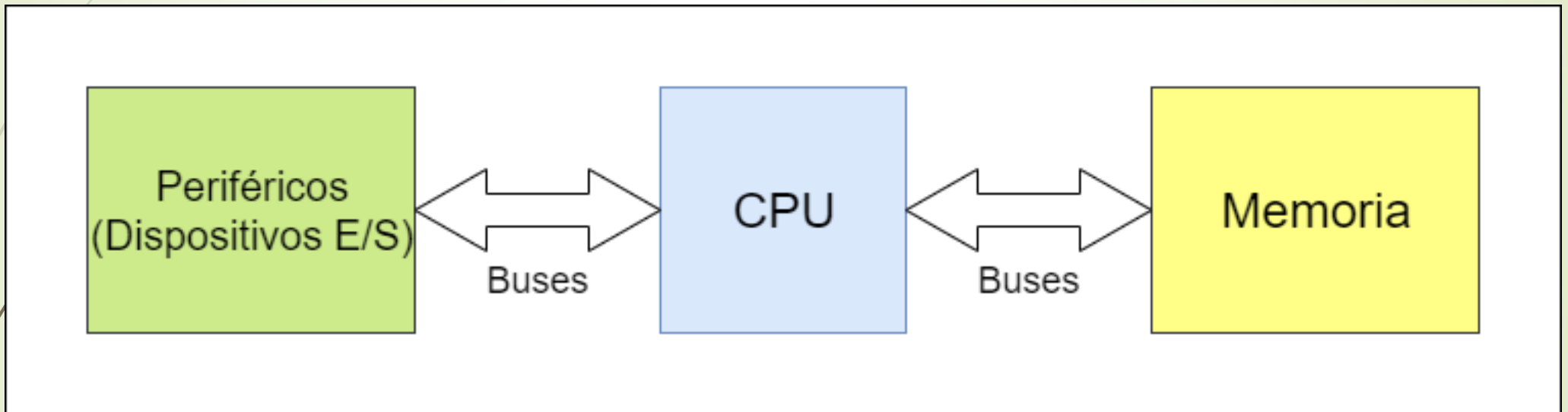
Objetivo: El alumno explicará los conceptos que le permitan analizar funcionalmente los distintos elementos, tanto de software como de hardware, que constituyen una computadora como repercusión en las características de operación del sistema.

1.1: Bloques funcionales de una computadora

- ¿Qué es una computadora?
- Es una **máquina programable** capaz de **realizar cálculos y operaciones** basados en una **secuencia de instrucciones bien definidas y entradas**, que sirven para obtener un **resultado**, además de que puede **almacenar información**.
- 4 Elementos:
 - Cálculos y operaciones
 - Secuencia de instrucciones (algoritmos)
 - Tiene entradas y salidas
 - Puede almacenar información

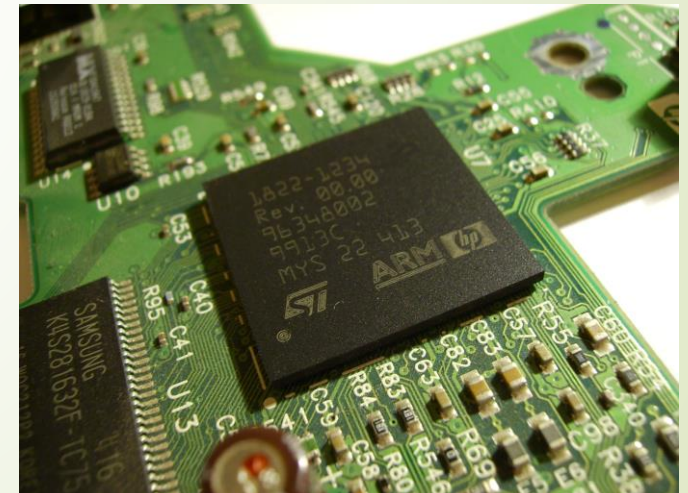


Bloques funcionales



Bloques funcionales

- **CPU (Unidad Central de Procesamiento):** Se considera como el núcleo de la computadora, es la unidad que se encarga de realizar la mayoría de los cálculos y operaciones que tienen lugar dentro de la computadora. También lo conocemos como **microprocesador**, o simplemente **procesador**.



Bloques funcionales

- **Periféricos (Dispositivos de Entrada/Salida):** Son elementos de la computadora que son externos a su núcleo central (CPU) que proporcionan entradas y salidas a una computadora. Son los que permiten la interacción humano-máquina.

Ejemplos

Entrada

- Teclado
- Mouse
- Touchscreen y touchpad
- Unidad de Disco

Salida

- Monitor
- Impresora
- Bocinas (audio)

Entrada/Salida

- Memorias externas
- Dispositivos USB
- Dispositivos Ethernet (router, switches, etc.)



Bloques funcionales

- **Memoria:** Es un dispositivo capaz de almacenar información de manera temporal (RAM) o permanente (ROM).

Tipos de memoria:

Memoria RAM

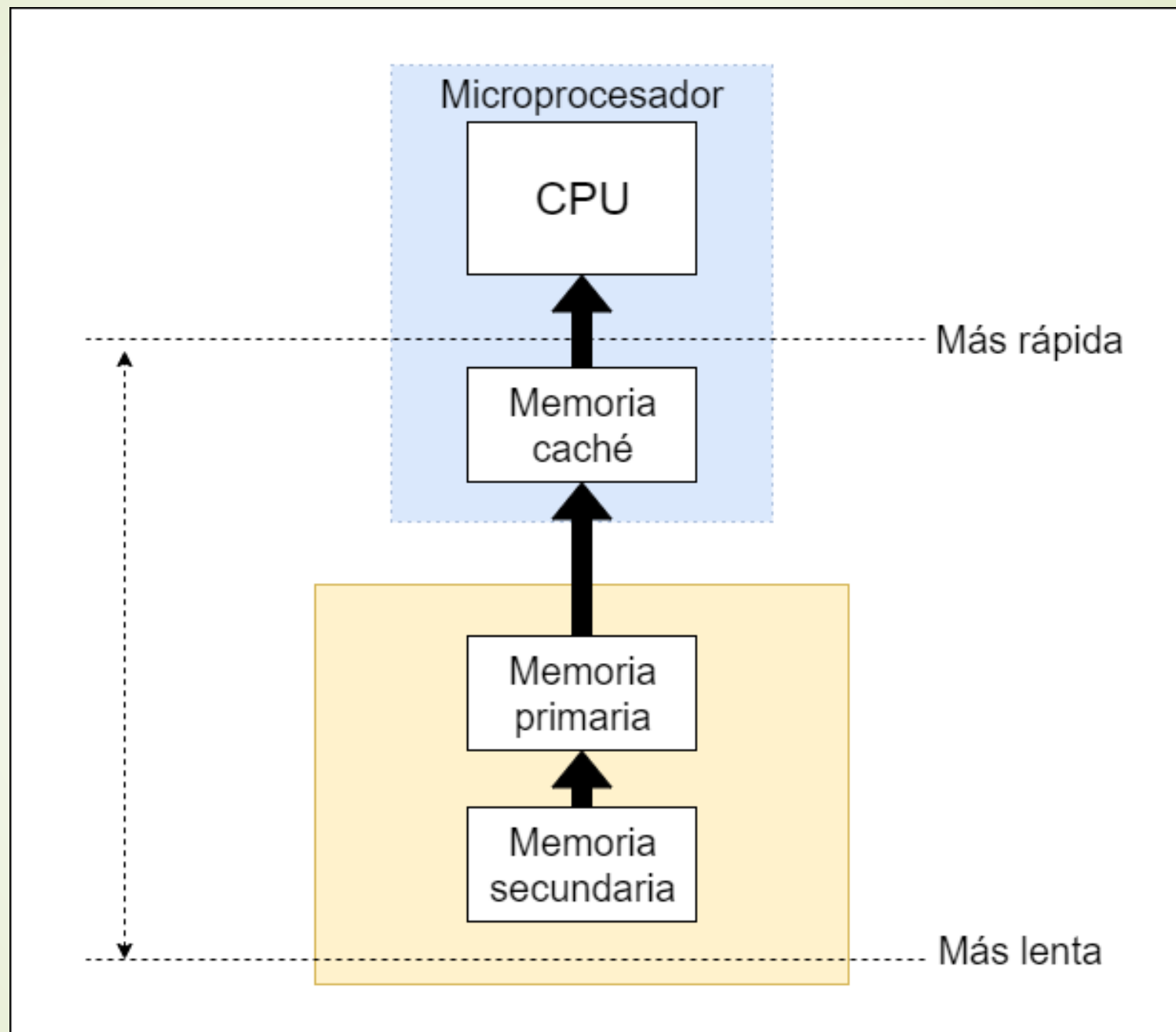
Almacena información que está siendo accedida por la CPU, de manera temporal.

Memoria ROM

Almacena información a largo plazo que se requiere para el funcionamiento del sistema.

Memoria Caché

Es un componente que almacena información frecuentemente solicitada para disponibilidad inmediata.



1.2 Bloques funcionales de un procesador

- ▶ La CPU **es un circuito integrado**

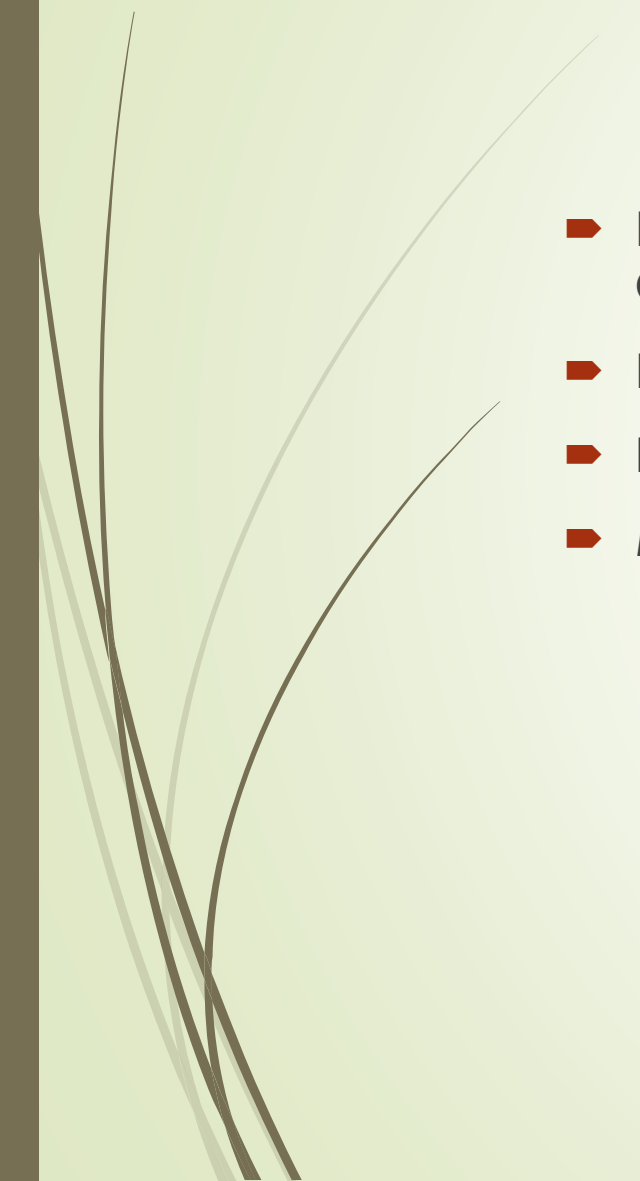
- ▶ Intel
- ▶ AMD
- ▶ ARM
- ▶ MicroChip
- ▶ Texas Instruments
- ▶ Qualcomm

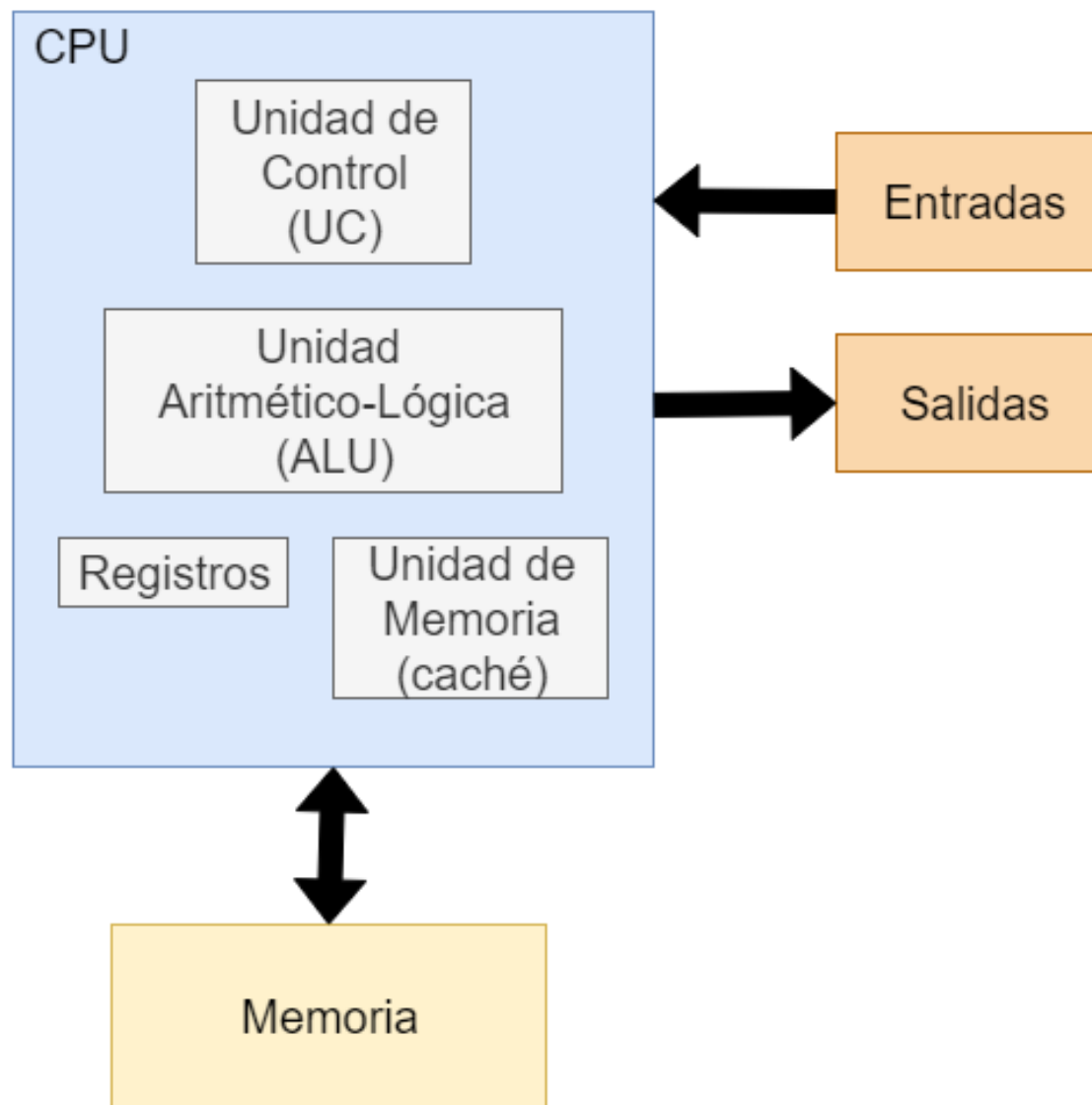
- ▶ La **velocidad de una CPU depende de la frecuencia de reloj**. La frecuencia se mide en Hertz (Hz)

$$1 \text{ Hz} = \frac{1}{s} \Rightarrow 3 \text{ GHz} = 3,000,000,000 \text{ ciclos/s}$$



Bloques funcionales de un procesador

- El reloj de la CPU se utiliza para **sincronizar todas las operaciones de la CPU.**
 - La CPU **ejecuta programas** (secuencia de instrucciones almacenados).
 - **Ejecuta operaciones aritméticas y manipulación de datos.**
 - Mantiene **información e instrucciones en uso.**
- 



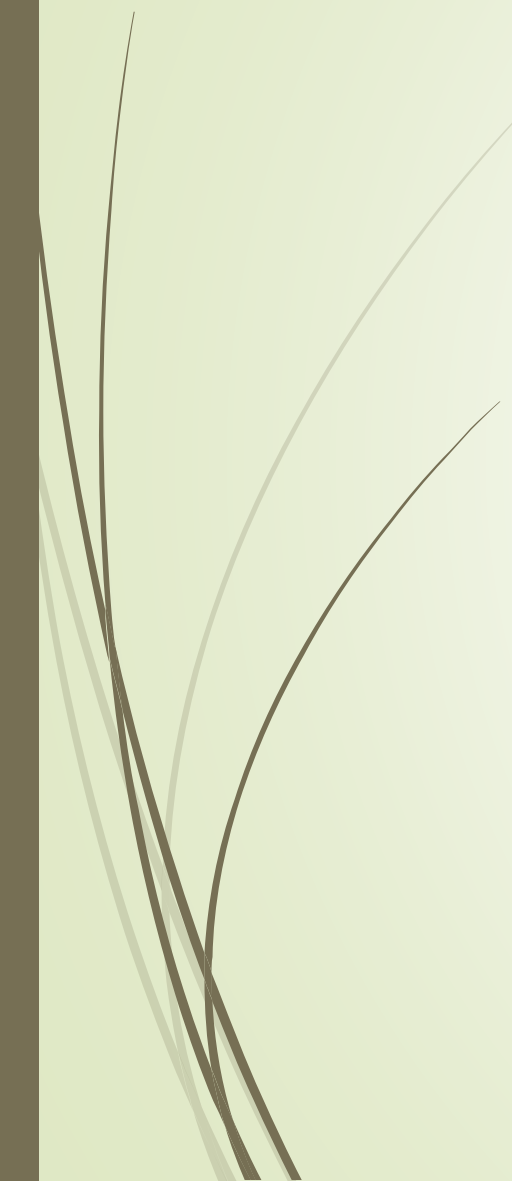


Bloques funcionales de un procesador

- **Unidad de control (UC):** indica y controla la comunicación dentro de la CPU.
 - Decide qué **circuitos dentro de la CPU se encuentran activos**.
 - **Coordina** la operación del **ciclo de instrucción** (búsqueda-interpretación-ejecución)
- **Unidad Aritmético-Lógica (ALU):** realiza varias **operaciones aritméticas** como adición (suma), sustracción (resta), multiplicación y división; y **operaciones lógicas** como AND, OR, NOT, XOR.
- **Registros:** son **unidades de memoria volátil** que son de **alta velocidad** pero de **poca capacidad**, sirven para guardar estados del procesador, así como datos temporales utilizados dentro de un programa.



Arquitectura de la computadora de acuerdo con la disposición de sus bloques funcionales

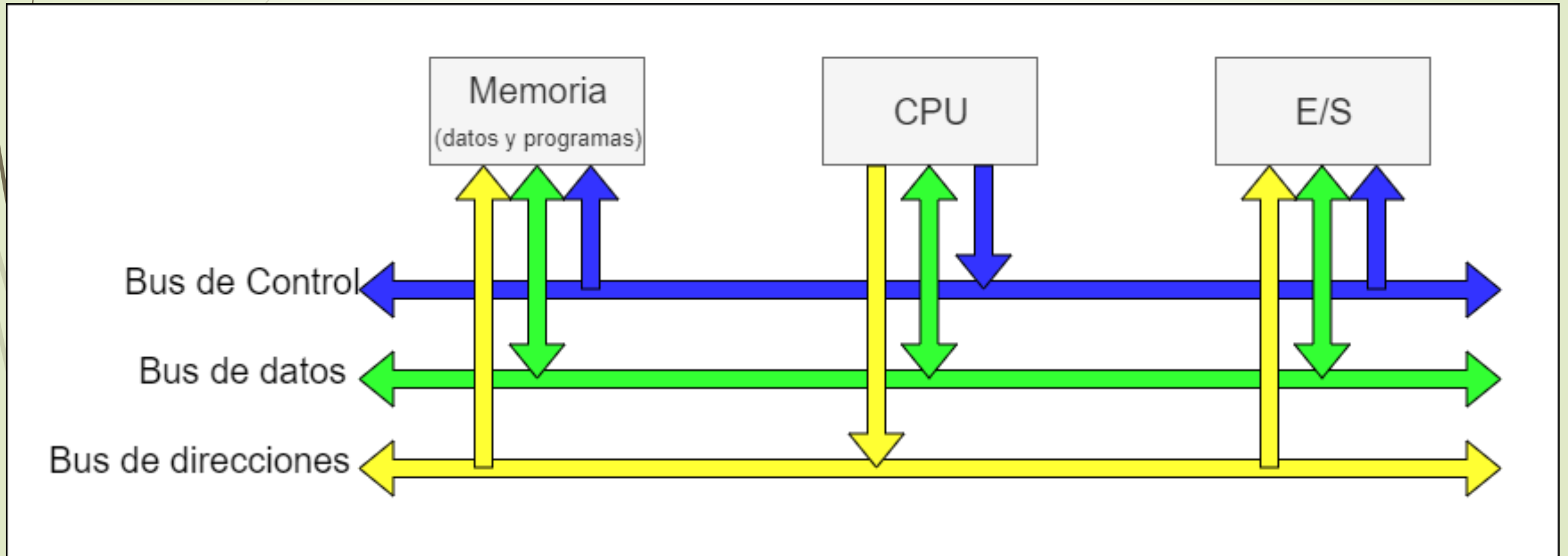
- Arquitectura von Neumann
 - Arquitectura Harvard
- 



Arquitectura von Neumann

- Es una arquitectura de computadoras diseñada por John von Neumann, **es un diseño para una computadora con un programa almacenado** y que es la base de las computadoras modernas.
- Consiste de **CPU, memoria y entradas/salidas**; además de **buses: de control, de datos y de direcciones**.
- Una característica muy importante es que en esta arquitectura **la memoria se utiliza** tanto **para guardar programas** como **para guardar datos**.

Arquitectura von Neumann

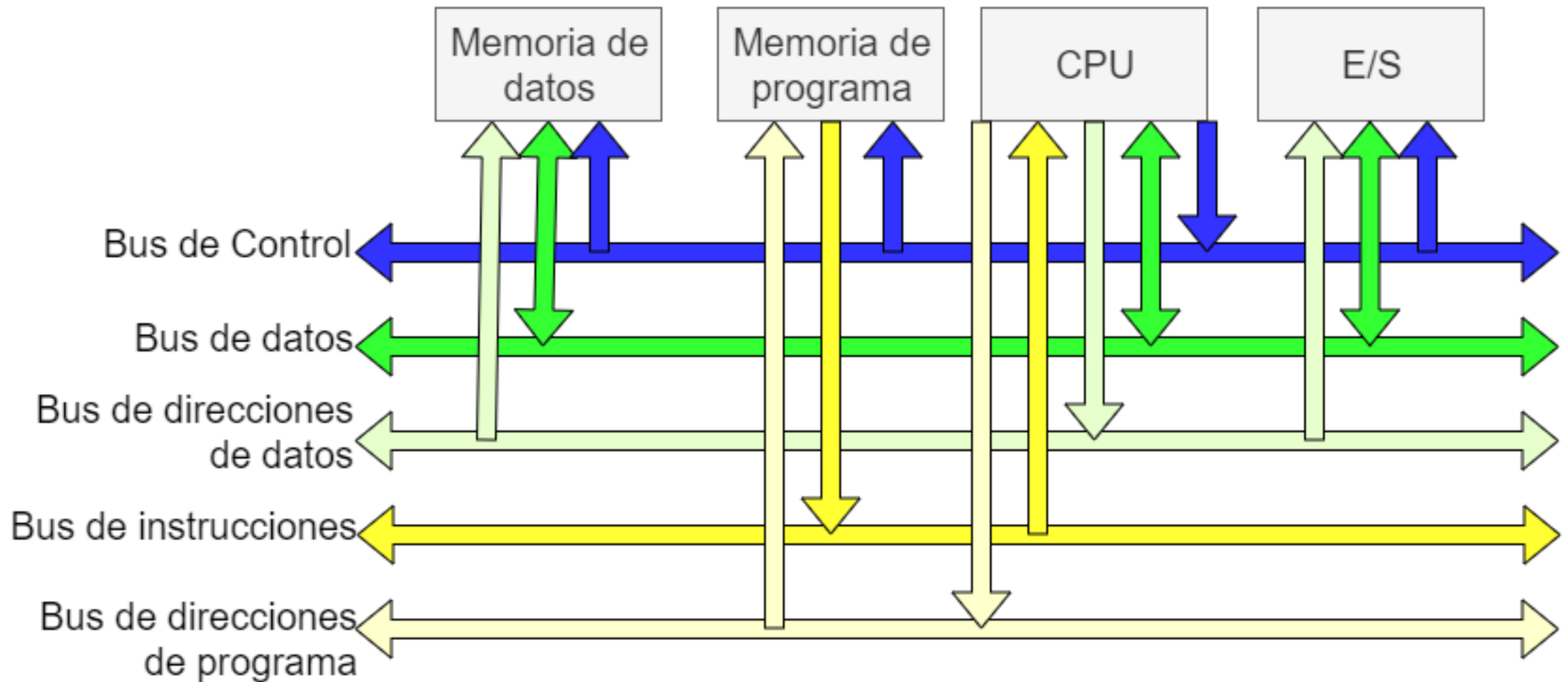




Arquitectura Harvard

- Es una arquitectura de computadoras cuyo término proviene de la computadora Harvard Mark I, creada en la Universidad de Harvard.
- Físicamente, **la memoria** se separa **en 2 bloques: memoria de datos y memoria de programa**. Los **programas se almacenan en memoria ROM** y los **datos se almacenan en memoria RAM**.
- Esta arquitectura **separa los buses** para **transferencia de datos y búsquedas de instrucciones**.
- Se utiliza, principalmente, en **dispositivos embebidos** como microcontroladores, procesadores de señales de audio y de video.

Arquitectura Harvard



Arquitectura von Neumann

➤ Ventajas:

- **Datos y programa agrupados** en el mismo **bloque de memoria**.
- **Instrucciones simplificadas**.
- **Simple uso de memoria**.
- Utiliza **menos hardware**.

➤ Desventajas:

- Procesamiento de **instrucciones de manera secuencial**.
- Sólo se puede ejecutar **una instrucción a la vez**, esto puede provocar un **cuello de botella**.
- Dado que se comparte la memoria para datos y programas, existe el **riesgo de escribir la sección de memoria correspondiente a un programa**.
- Depende de **la velocidad del bus de datos**.

Arquitectura Harvard

➤ Ventajas:

- Puede hacer **búsqueda de datos e instrucciones de manera paralela**.
- La **corrupción del programa es menos probable** debido a que **datos e instrucciones viajan a través de buses separados**.

➤ Desventajas:

- La **memoria de datos libre no puede usarse para programa**; y viceversa, la **memoria de programa libre no puede usarse para datos**.
- La **producción de una computadora** con esta arquitectura **puede ser más cara** porque **utiliza más hardware**.



1.3 Ciclo de instrucción (búsqueda-decodificación-ejecución)

- Es el **periodo que tarda el procesador en realizar o ejecutar una instrucción en código máquina.**
- Prácticamente, está compuesto de **3 etapas:**
 - **Búsqueda**
 - **Decodificación**
 - **Ejecución**
- El ciclo de instrucción puede tener etapas adicionales, dependiendo de la instrucción.



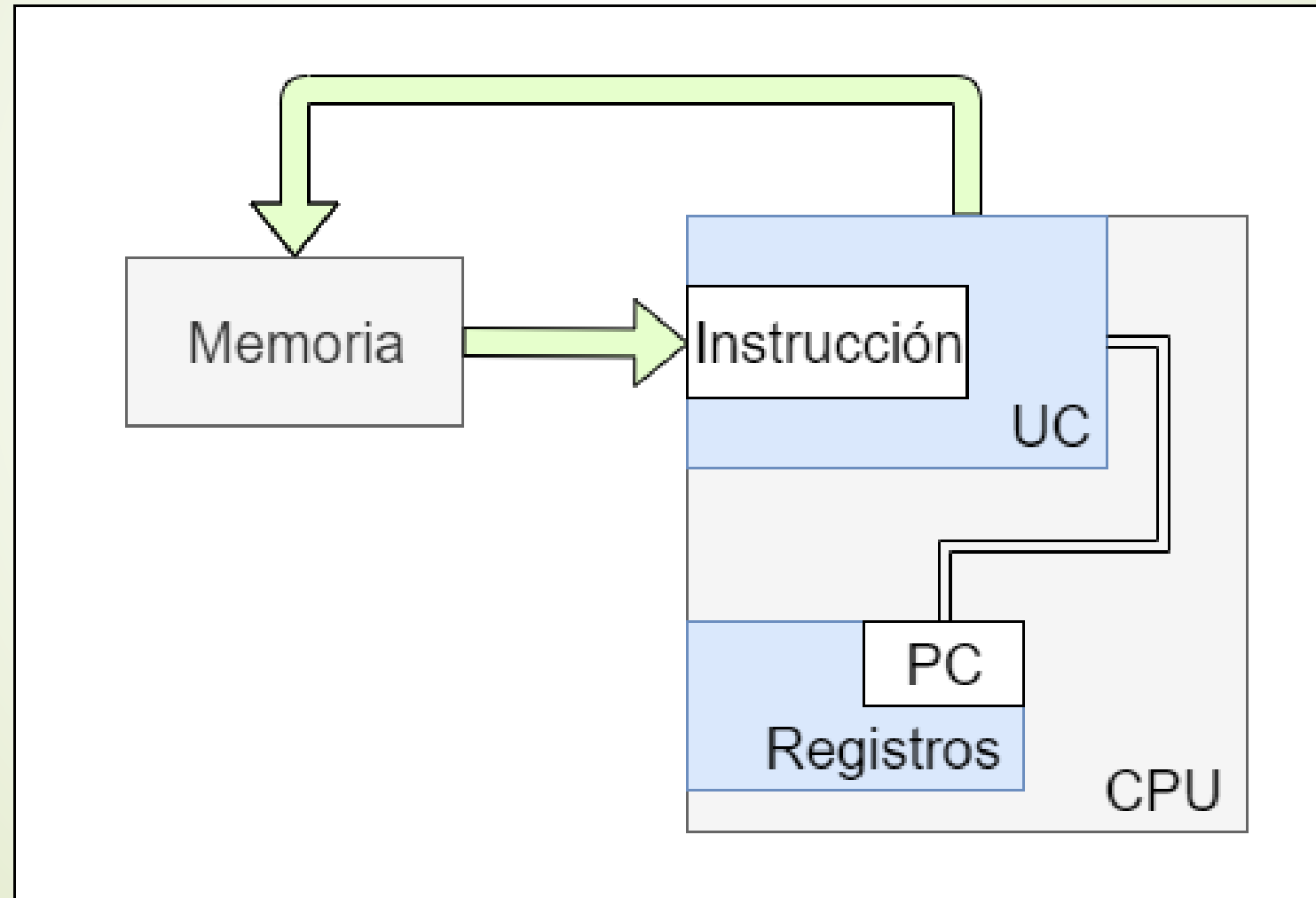
Ciclo de instrucción

Búsqueda

- ▶ La **instrucción** a ejecutar **se carga desde una dirección de memoria** que se **almacena en el contador de programa (PC)** y se **almacena la instrucción** en el **Registro de instrucción** (dentro de la unidad de control). Al final, **el PC apunta a la siguiente instrucción** que será leída en el siguiente ciclo.

Ciclo de instrucción

Búsqueda





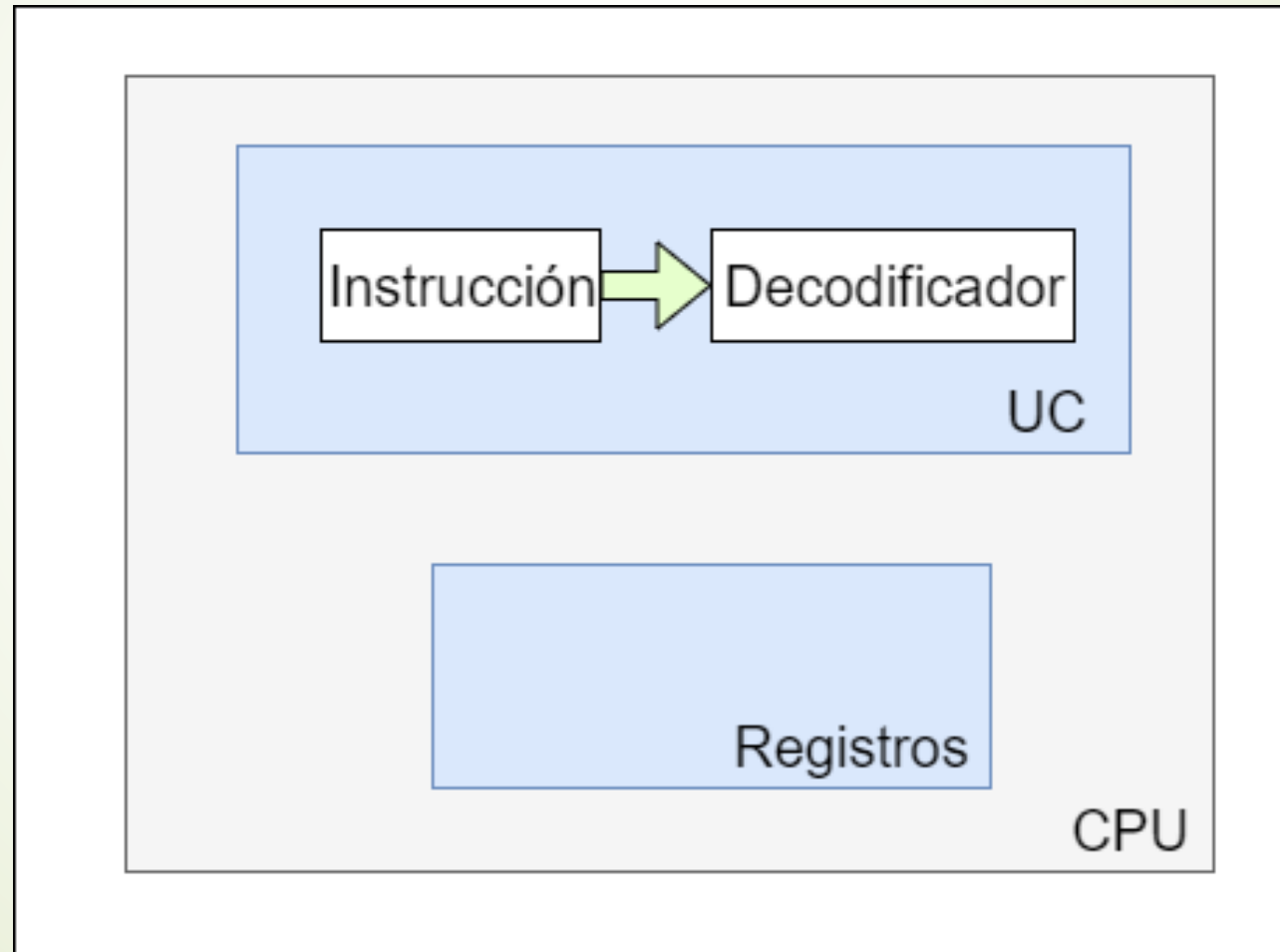
Ciclo de instrucción

Decodificación

- Durante esta etapa, la **instrucción del registro de instrucciones es interpretada** por el **decodificador** del procesador.

Ciclo de instrucción

Decodificación





Ciclo de instrucción

Búsqueda de datos

- Dependiendo de la instrucción, **se puede requerir una etapa adicional** a la ejecución de dicha instrucción.
- Si la instrucción **accede a memoria de datos** éstos se obtendrán en el siguiente ciclo de reloj, si no lo hace, el siguiente paso será el de ejecución.



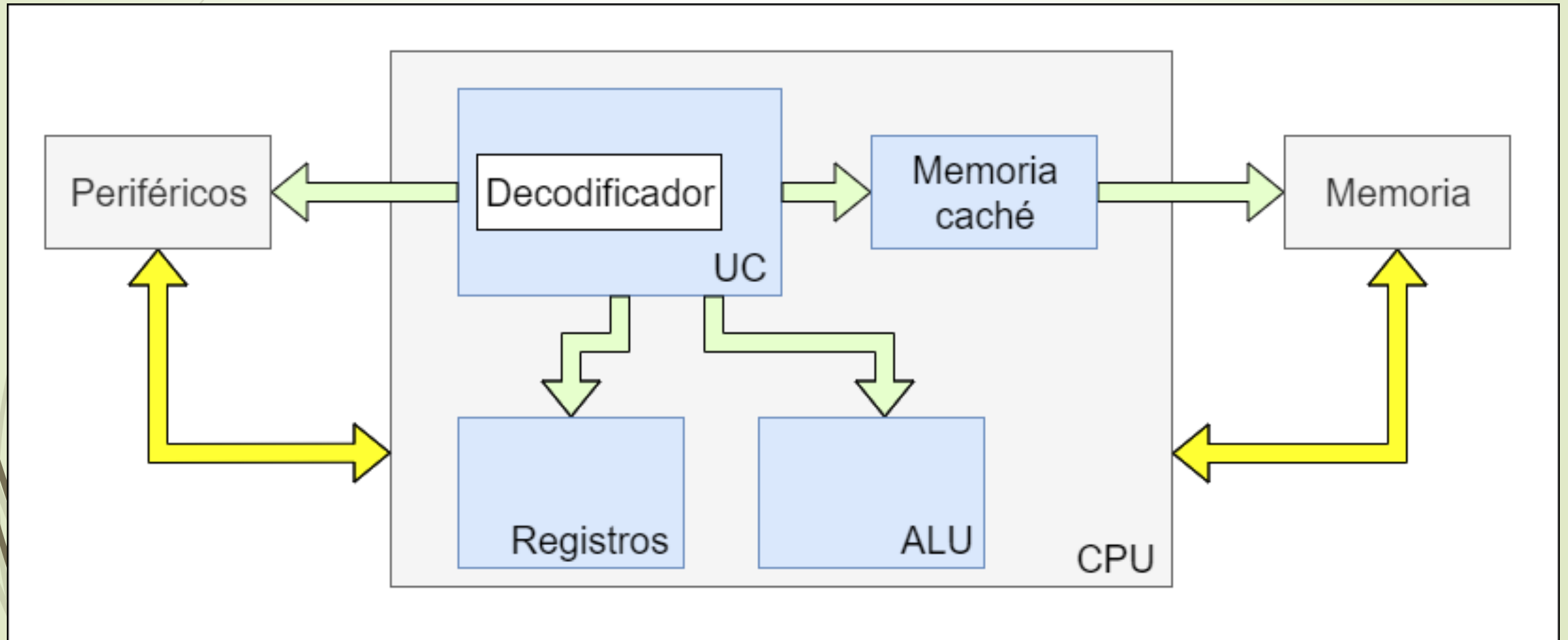
Ciclo de instrucción

Ejecución

- La **unidad de control** transmite las **señales** necesarias a las unidades dentro y fuera de la CPU para **realizar las acciones** requeridas por la instrucción. Si la instrucción requiere un **salto a otra dirección de memoria**, el **PC se actualiza** con la dirección de memoria del salto.

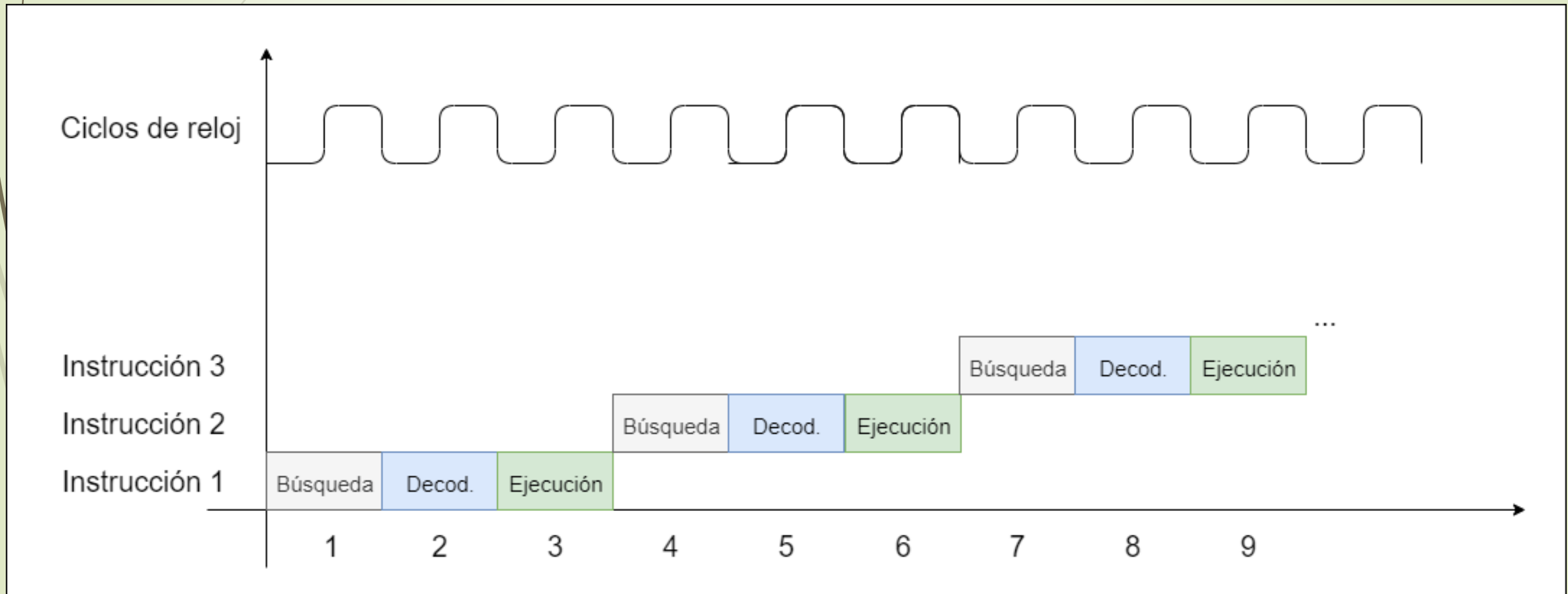
Ciclo de instrucción

Ejecución

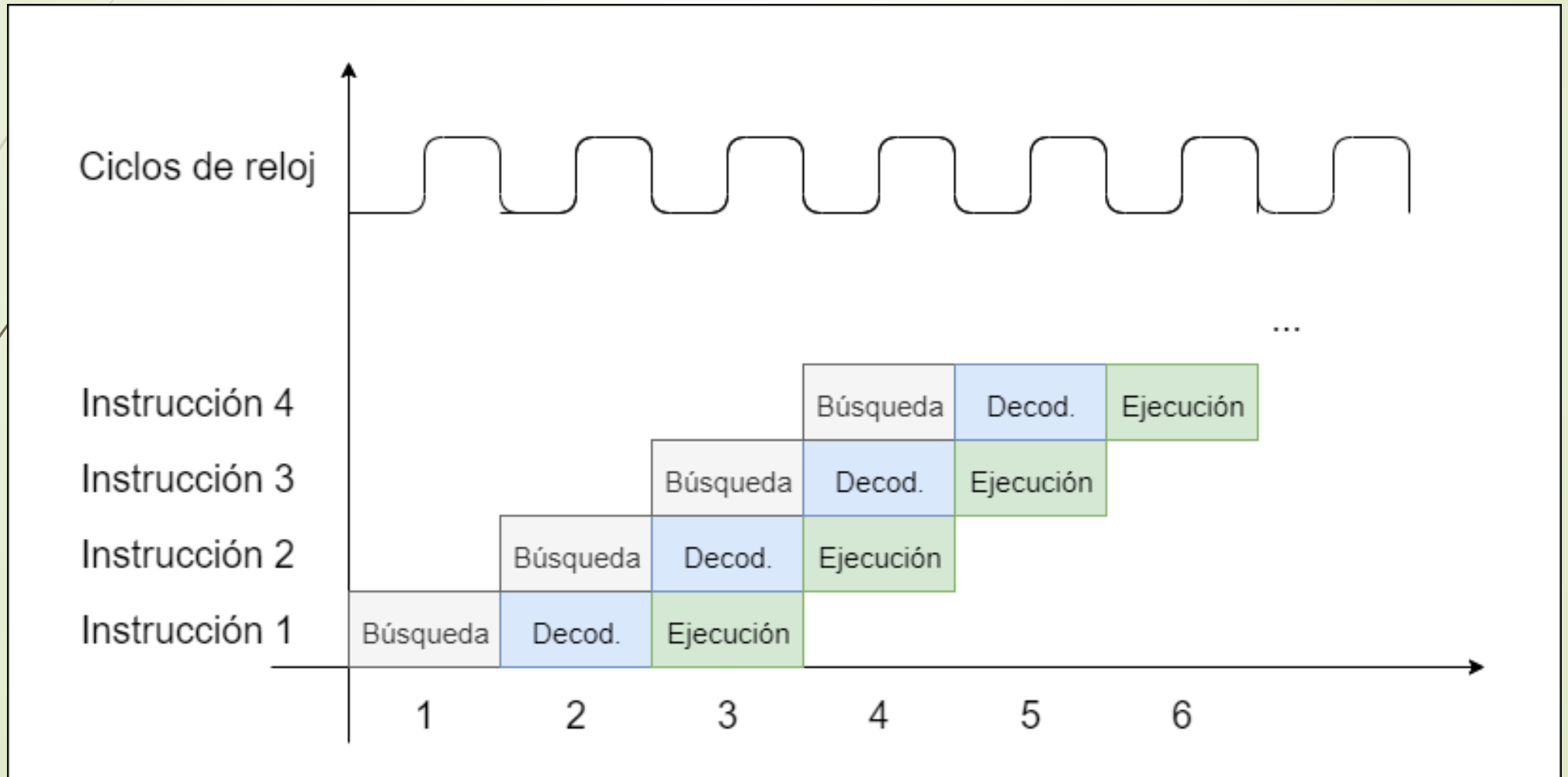


Ciclo de instrucción

Arquitectura von Neumann



Ciclo de instrucción Arquitectura Harvard



1.4 Esquemas de direccionamiento

- Una **máquina realiza tareas** con base en las **instrucciones proporcionadas en su conjunto de instrucciones**.
- El **conjunto de instrucciones** es la **lista de instrucciones que la CPU puede entender y ejecutar**. Depende de:
 - Tipos de datos
 - Registros
 - Modos de direccionamiento
- Una **instrucción**, en código máquina, está representada por **1s y 0s**.

Esquemas de direccionamiento

- El **formato de una instrucción** está, más o menos, dado de la siguiente manera:

Código de operación	Operando 1	Operando 2	Operando 3	...	Operando M
---------------------	------------	------------	------------	-----	------------

- La longitud de una instrucción** (cantidad de bits) puede variar dependiendo el **número de direcciones que contiene**.
- Una dirección es un operando en la instrucción, pero un operando no necesariamente es una dirección (podría ser un valor constante).



Esquemas de direccionamiento

- La **organización de la CPU** puede ser de **3 tipos** basado en el **número de campos de direcciones** dentro de la instrucción:
 - 1. Organización de Acumulador sencillo:** la operación se realiza involucrando **un registro** especial llamado **acumulador**.
 - 2. Organización de Registros Generales:** la operación se realiza sobre **múltiples registros**.
 - 3. Organización de Pila:** la operación se realiza **sobre el stack (pila)**, NO contiene campos de dirección.
- En una máquina, **no necesariamente se aplica una única organización**. En la **actualidad**, en los procesadores se aplican **mezclas de varias organizaciones**.



Esquemas de direccionamiento

- Basado en el **número de direcciones**, las instrucciones se pueden clasificar como:
 - Instrucciones de 0 direcciones
 - Instrucciones de 1 dirección
 - Instrucciones de 2 direcciones
 - Instrucciones de 3 direcciones

Instrucciones de 0 direcciones

- Una máquina basada en pila/stack **no utiliza campo de dirección en la instrucción**.
- En este caso, las operaciones se evalúan convirtiéndolas a **notación polaca inversa** o notación **posfija**.

Ejemplo:

$X = (A+B)*(C+D)$ -----> Expresión original

Instrucciones de 0 direcciones

$X = AB + CD + *$ -----> Notación posfija

PUSH A	TOP = A
PUSH B	TOP = B
ADD	TOP = A+B
PUSH C	TOP = C
PUSH D	TOP = D
ADD	TOP = C+D
MUL	TOP = (A+B) * (C+D)
POP X	X = TOP

Instrucciones de 1 dirección

- Implica un **registro acumulador** para la manipulación de datos. Un operando es un acumulador y el otro está en un **registro o localidad de memoria** (la CPU sabe que un operando está en el registro acumulador así que no es necesario especificar).

Código de operación	Operando
---------------------	----------

Ejemplo:

$X = (A+B)*(C+D)$ -----> Expresión original

Instrucciones de 1 dirección

$X = (A+B)*(C+D)$ -----> Expresión original

LOAD A	$ACC = A$
ADD B	$ACC = ACC + B$
STORE T	$T = ACC$
LOAD C	$ACC = C$
ADD D	$ACC = ACC + D$
MUL T	$ACC = ACC * T$
STORE X	$X = ACC$

Instrucciones de 2 direcciones

- Éste es el esquema más común en las computadoras comerciales. En éste, **2 direcciones** pueden especificarse **en la instrucción**.
- A diferencia del esquema de 1 dirección, en el esquema de 2 direcciones **el resultado se almacena en una localidad diferente que el acumulador**. Sin embargo, el formato de instrucción requiere mayor cantidad de bits.

Código de operación	Destino	Origen
---------------------	---------	--------

Ejemplo:

$X = (A+B)*(C+D)$ -----> Expresión original

Instrucciones de 2 direcciones

$X = (A+B)*(C+D)$ -----> Expresión original

MOV R1,A

$R1 = A$

ADD R1,B

$R1 = R1 + B$

MOV R2,C

$R2 = C$

ADD R2,D

$R2 = R2 + D$

MUL R1,R2

$R1 = R1 * R2$

MOV X,R1

$X = R1$

Instrucciones de 3 direcciones

- Éstas tienen **3 campos de dirección** para especificar **un registro o una localidad de memoria en cada uno**.
- Los **programas** creados con este esquema son **más cortos** pero requieren **instrucciones de mayor longitud**.
- Que los programas sean más cortos no quiere decir que sean más rápidos porque **cada microoperación se realizará en un ciclo de reloj**.

Código de operación	Destino	Operando 1	Operando 2
---------------------	---------	------------	------------

Ejemplo:

$X = (A+B)*(C+D)$ -----> Expresión original

Instrucciones de 3 direcciones

$X = (A+B)*(C+D)$ -----> Expresión original

ADD R1,A,B

$R1 = A + B$

ADD R2,C,D

$R2 = C + D$

MUL X,R1,R2

$X = R1 * R2$